

Bradley-Terry Scalable Model Simulation

R Environment Setup

This section installs and loads the necessary packages for the simulation.

```
# Installing from GitHub
# install.packages("devtools") # if required
devtools::install_github("EllaKaye/BradleyTerryScalable", build_vignettes = TRUE)
library(BradleyTerryScalable)
library(Matrix)
library(dplyr)
library(ggplot2)
```

Small-Scale Simulation

```
data(citations)
```

Transform the data to the btdata object the package can deal with

```
citations_btdata <- btdata(citations)
citations_btdata
```

```
## $wins
## 4 x 4 sparse Matrix of class "dgCMatrix"
##           citing
## cited      Biometrika Comm Statist JASA JRSS-B
## Biometrika      714      730  498   221
## Comm Statist      33      425   68    17
## JASA      320      813 1072   142
## JRSS-B      284      276  325   188
##
## $components
## $components$`1`
## [1] "Biometrika" "Comm Statist" "JASA" "JRSS-B"
##
##
## attr(,"class")
## [1] "btdata" "list"
```

Checking connectivity

```
summary(citations_btdata)
```

```
## Number of items: 4  
## Density of wins matrix: 1  
## Fully-connected: TRUE
```

Since it is fully connected, we use $a=1$ in the `btfit` function (Maximum Likelihood Estimate)

```
citations_fit <- btfit(citations_btdata, 1)
```

Checking the info for the model

```
summary(citations_fit)
```

```
## Warning: 'data_frame()' was deprecated in tibble 1.1.0.  
## i Please use 'tibble()' instead.  
## i The deprecated feature was likely used in the BradleyTerryScalable package.  
## Please report the issue at  
## <https://github.com/EllaKaye/BradleyTerryScalable/issues>.  
## This warning is displayed once every 8 hours.  
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was  
## generated.
```

```
## $call  
## btfit(btdata = citations_btdata, a = 1)  
##  
## $item_summary  
## # A tibble: 4 x 3  
##   component      item      estimate  
##   <chr>         <chr>         <dbl>  
## 1 full_dataset JRSS-B           1.06  
## 2 full_dataset Biometrika      0.790  
## 3 full_dataset JASA             0.310  
## 4 full_dataset Comm Statist    -2.16  
##  
## $component_summary  
## # A tibble: 1 x 4  
##   component      num_items iters converged  
##   <chr>          <int> <int> <lgl>  
## 1 full_dataset      4      2 TRUE
```

Result Interpretation

`$call` (Note: R code used to execute the model fitting)

`$item_summary` (Data frame with fit results for each item)

Note: `estimate` is the estimated strength parameter on the log scale.

Note: Higher value = higher strength/rank

`$component_summary` (Note: Data frame summarizing the status of the fitted components)

Note: `num_items` is the count of items in that component

Note: `iters` = iterations for convergence

Now we do a simulation using the fitted model, 100 times, and each is independent

```
citations_sim <- simulate(citations_fit, nsim = 100, seed = 1)
```

Display some results from the simulation (first two simulated wins matrices)

```
citations_sim[1:2]
```

```
## $sim_1
## 4 x 4 sparse Matrix of class "dgCMatrix"
##           citing
## cited      JRSS-B Biometrika JASA Comm Statist
## JRSS-B      .          286  316          285
## Biometrika  219          .  495          725
## JASA        151        323  .          817
## Comm Statist 8          38  64          .
##
## $sim_2
## 4 x 4 sparse Matrix of class "dgCMatrix"
##           citing
## cited      JRSS-B Biometrika JASA Comm Statist
## JRSS-B      .          291  315          273
## Biometrika  214          .  511          723
## JASA        152        307  .          811
## Comm Statist 20          40  70          .
```

Large-Scale Tournament Simulation

IMPORTANT NOTE: They do not have real tournament data; they build a synthetic dataset that mimics real tournament data characteristics.

```
set.seed(1989)
```

Setting number of players (items)

```
n_items <- 1000
```

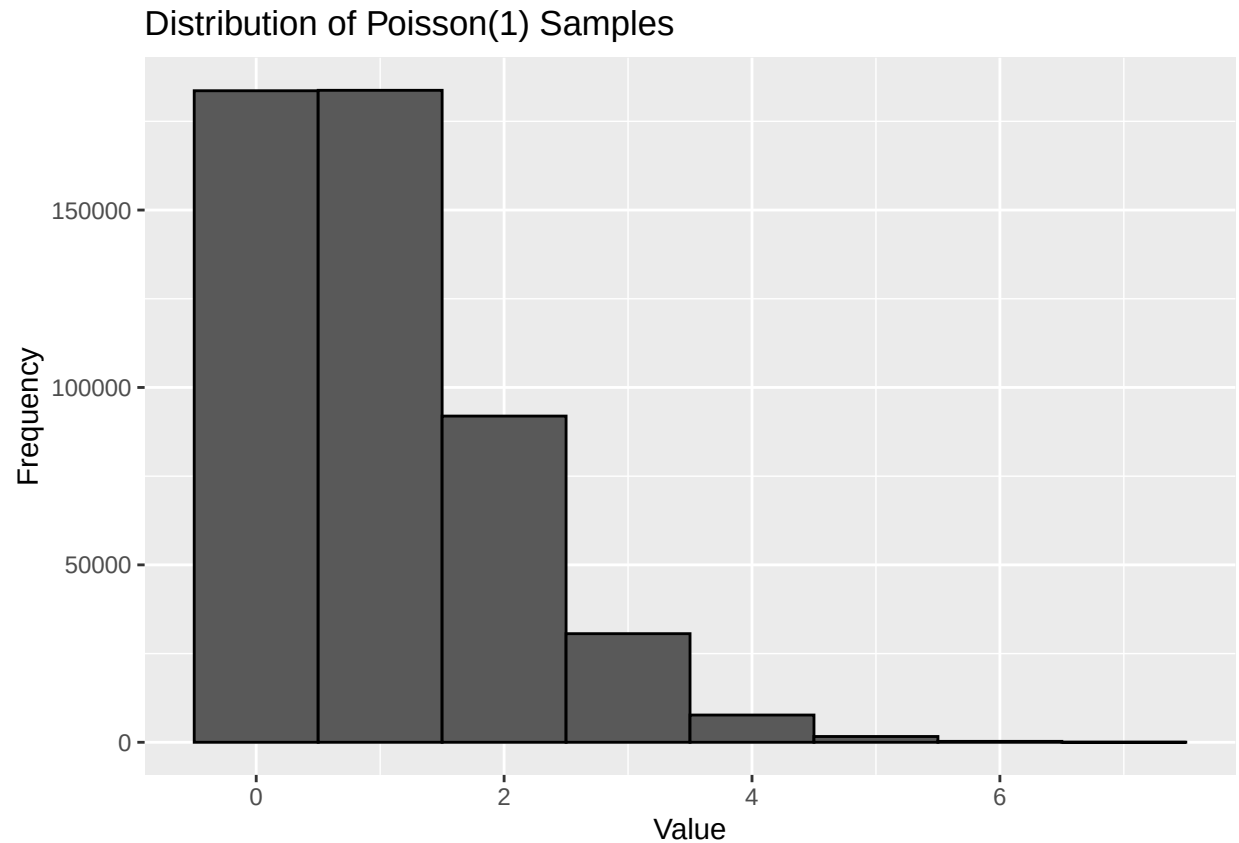
1. Generating the Sparse Comparison Totals (Nmatrix)

Create a Poisson distribution with $\lambda=1$ to simulate the total number of comparisons between 1000 players.

The formula calculates the number of unique pairs: $n * (n - 1) / 2$

```
Nvalues <- rpois(n = n_items * (n_items - 1) / 2, lambda = 1)
poisdis <- data.frame(Nvalues = Nvalues)

ggplot(poisdis, aes(x = Nvalues)) +
  geom_histogram(binwidth = 1, color = "black") +
  labs(title = "Distribution of Poisson(1) Samples",
       x = "Value", y = "Frequency")
```



logic function for later use

```
notzero <- Nvalues > 0
```

build a empty matrix

```
Nmatrix_empty <- Matrix(nrow = n_items, ncol = n_items)
```

Identify the element which is not 0, later we want to fill into the matrix

```
ij <- which(lower.tri(Nmatrix_empty), arr.ind = TRUE)[notzero, ]
```

fill in the matrix with the generated data

```
Nmatrix <- sparseMatrix(  
  i = ij[, 1],  
  j = ij[, 2],  
  x = Nvalues[notzero],
```

```

symmetric = TRUE,
dims = c(n_items, n_items)
)

```

now for the BT part

generating “hidden ability value” for each player, assuming it is normally distributed

and we divide by 4 to reduce the sd of generated data

```

pi_vec <- exp(rnorm(n_items) / 4)

```

now we normalize pi_vec so its mean is 1, eliminate parameter redundancy caused by the Bradley-Terry model’s scale invariance

```

pi_vec <- pi_vec / mean(pi_vec)

```

juz like the small data set, we do a simulation

given the “hidden ability value” and “the number of time they played to each other”

```

big_matrix <- simulate_BT(pi_vec, Nmatrix, nsim = 1, seed = 1)[[1]]

```

```

## 'as(<dsCMatrix>, "dgCMatrix")' is deprecated.
## Use 'as(., "generalMatrix")' instead.
## See help("Deprecated") and help("Matrix-deprecated").

```

convert the resulting matrix into a btdata object(the form this package can deal with)

```

big_btdata <- btdata(big_matrix)

```

check if it is fully connected

```

summary(big_btdata)

```

```

## Number of items: 1000
## Density of wins matrix: 0.390967
## Fully-connected: TRUE

```

fit the model using MLE (a=1)

```
the_model <- btfit(big_btdata, a = 1)
```

extract the fitted ability parameters (pi)

```
pi_fitted <- the_model$pi$full_dataset
```

plot

```
plot_df <- tibble(  
  x = log(pi_vec[as.numeric(names(pi_fitted))]),  
  y = log(pi_fitted)  
)  
  
ggplot(plot_df, aes(x, y)) +  
  geom_point(alpha = 0.5) +  
  geom_abline() +  
  xlab("true strength") +  
  ylab("maximum likelihood estimate") +  
  ggtitle("1000-player simulation from a Bradley-Terry model") +  
  theme(plot.title = element_text(hjust = 0.5))
```

